

Diffie-Hellman Key Exchange Protocol

A Visual Guide to Cryptographic Key Establishment

Lecture Notes for Cybersecurity Students

Dr. Mohammed Tawfik

Assistant Professor of Cybersecurity and Cloud Computing
Ajloun National University, Jordan

`kmkhol01@gmail.com`

December 13, 2025

Learning Objectives

- Understand the mathematical foundation of Diffie-Hellman
- Visualize the key exchange process step-by-step
- Apply the protocol through practical examples
- Analyze security considerations and attack vectors
- Recognize real-world applications in modern cryptography

Contents

1	Introduction	3
1.1	The Problem Visualization	3
1.2	The Paint Mixing Analogy	3
2	Mathematical Foundation	4
2.1	Modular Arithmetic Visualization	4
2.2	The Discrete Logarithm Problem	4
2.3	Visual Comparison: Easy vs Hard	4
3	Protocol Visualization	5
3.1	Complete Protocol Flow	5
3.2	Information Flow Diagram	6
3.3	Security Visualization: What Eve Sees	6
4	Step-by-Step Algorithm	7
5	Mathematical Proof of Correctness	8
6	Worked Example 1: Small Numbers (Educational)	9
6.1	Visual Step-by-Step Execution	9
6.2	Detailed Calculation Breakdown	10
7	Worked Example 2: Moderate Size	12
8	Worked Example 3: Larger Prime	14
9	Computational Complexity Analysis	16
9.1	Performance Comparison Table	16
10	Security Analysis	16
10.1	Attack Scenarios	16
10.2	Security Requirements	17
11	Real-World Applications	18
11.1	Modern Cryptographic Protocols	18
11.2	TLS Handshake Visualization	18
12	Elliptic Curve Variant (ECDH)	19
12.1	Why Elliptic Curves?	19
12.2	Visual Comparison	19
13	Summary and Key Takeaways	20
14	Practice Problems for Students	21
15	Further Reading and Resources	21

1 Introduction

The **Diffie-Hellman Key Exchange** is a groundbreaking cryptographic protocol published by Whitfield Diffie and Martin Hellman in 1976. It revolutionized cryptography by solving a fundamental problem: *How can two parties establish a shared secret key over an insecure channel?*

1.1 The Problem Visualization

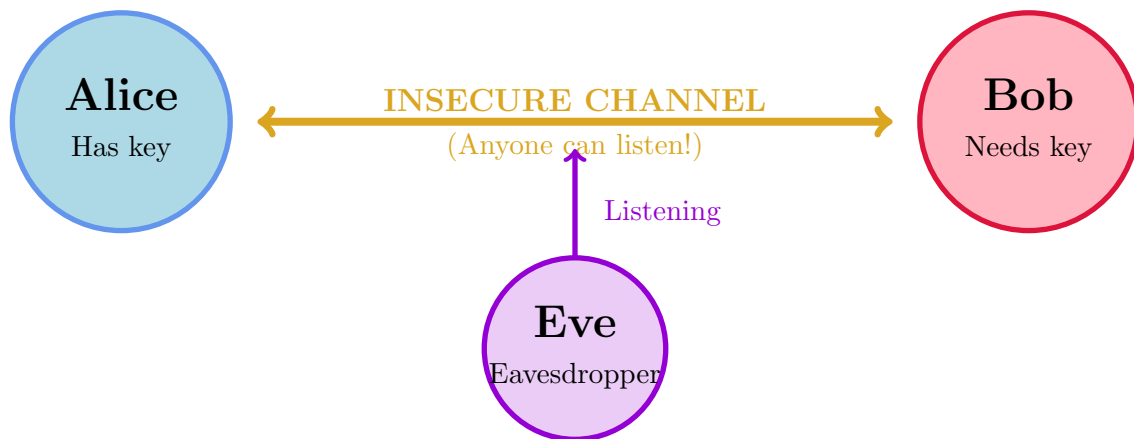


Figure 1: The Key Distribution Problem

1.2 The Paint Mixing Analogy

Before diving into mathematics, let's understand the concept through a visual analogy:

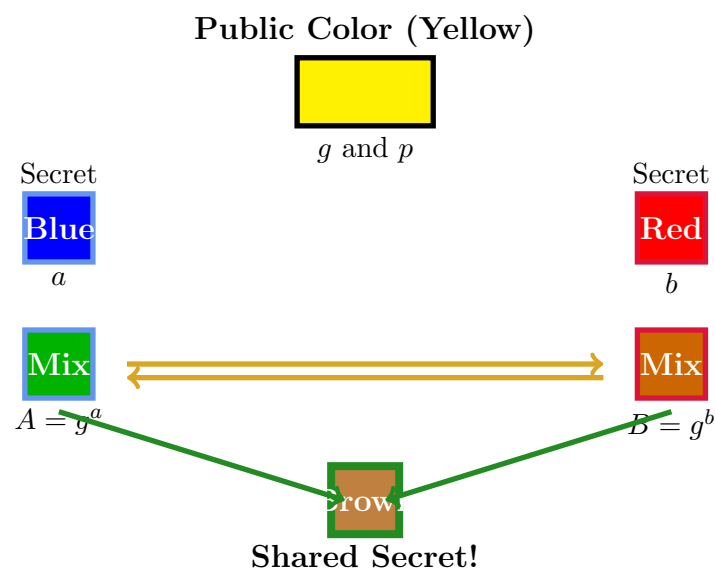


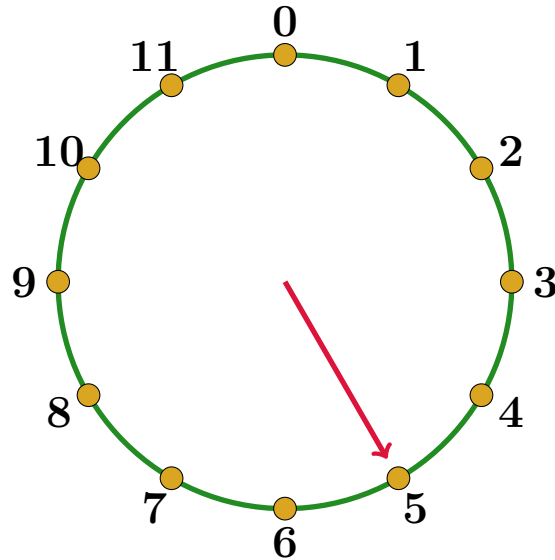
Figure 2: Paint Mixing Analogy for Diffie-Hellman

Key Insight: Just as it's easy to mix colors but hard to separate them, it's easy to compute $g^a \bmod p$ but hard to find a from the result!

2 Mathematical Foundation

2.1 Modular Arithmetic Visualization

Think of modular arithmetic as a clock. When we compute $a \bmod p$, we're finding the remainder after division by p .



Example: $17 \bmod 12 = 5$ (go around once, land on 5)

Figure 3: Modular Arithmetic as a Clock

2.2 The Discrete Logarithm Problem

Why is Diffie-Hellman Secure?

Easy Direction (Forward):

$$y = g^x \bmod p \quad \checkmark \text{ Fast to compute}$$

Hard Direction (Reverse):

Given y, g, p , find x such that $y = g^x \bmod p$ $\times$ Very difficult!

This asymmetry is the foundation of security!

2.3 Visual Comparison: Easy vs Hard

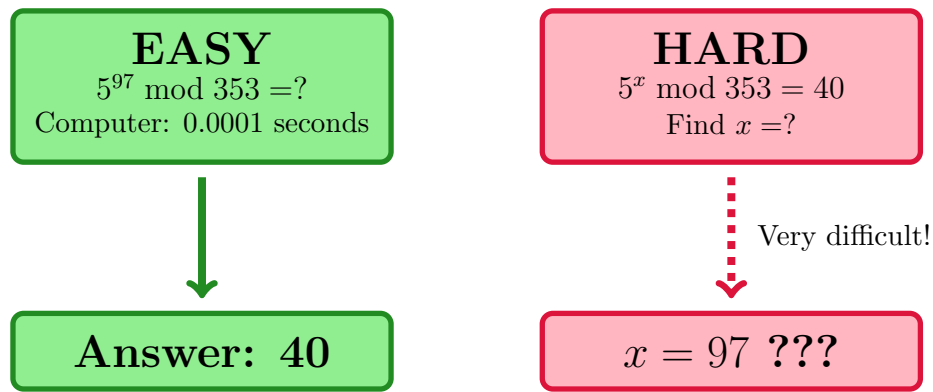


Figure 4: Computational Asymmetry in Diffie-Hellman

3 Protocol Visualization

3.1 Complete Protocol Flow

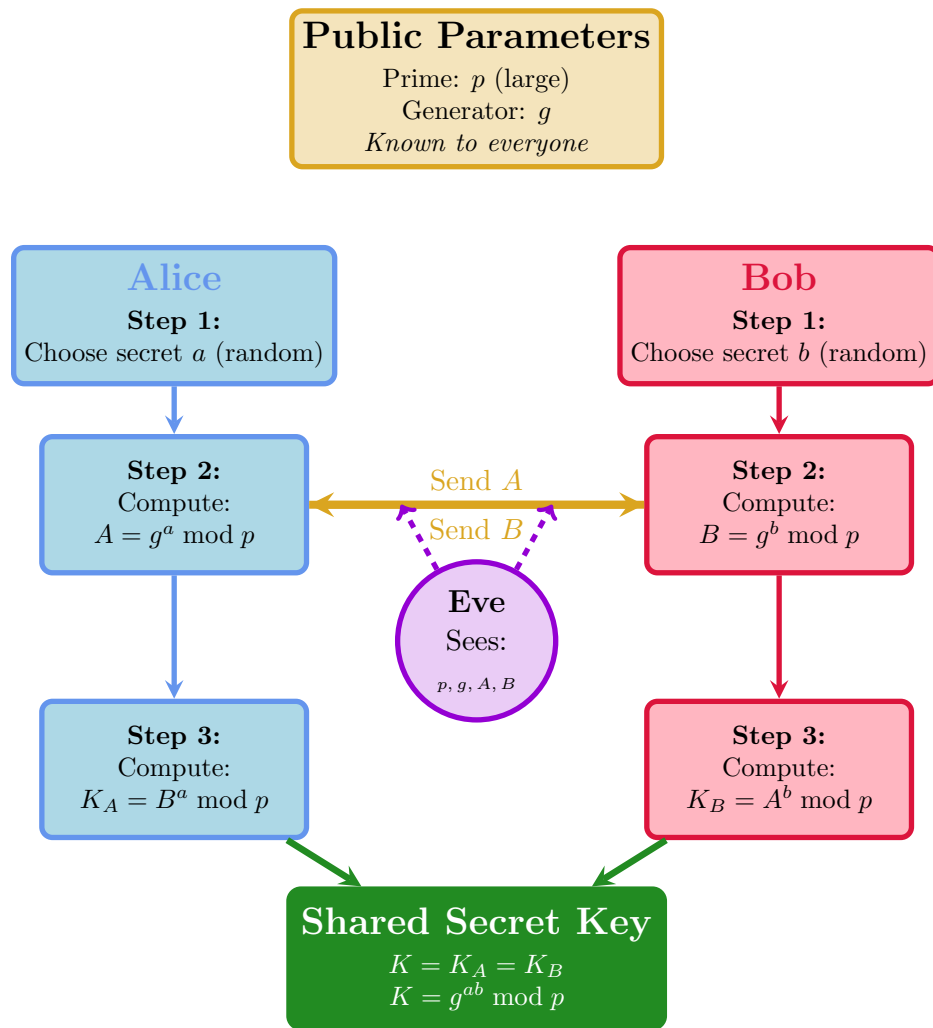


Figure 5: Complete Diffie-Hellman Protocol Flow

3.2 Information Flow Diagram

3.3 Security Visualization: What Eve Sees

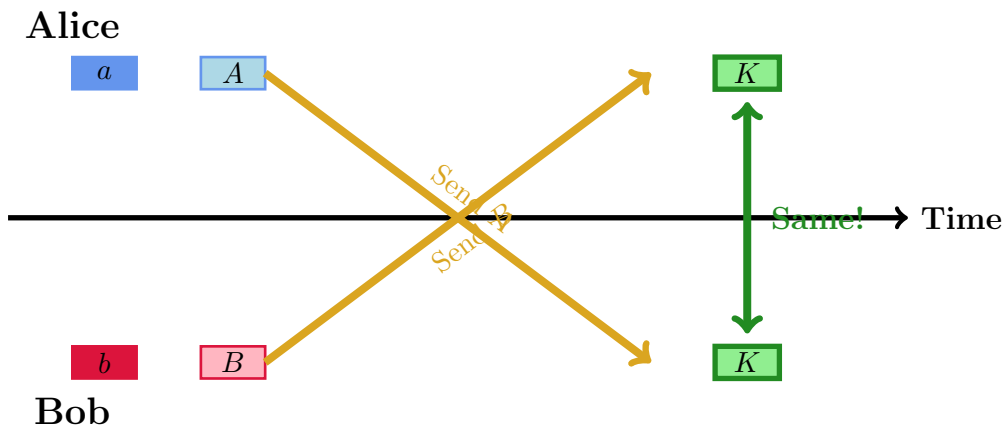


Figure 6: Information Exchange Timeline

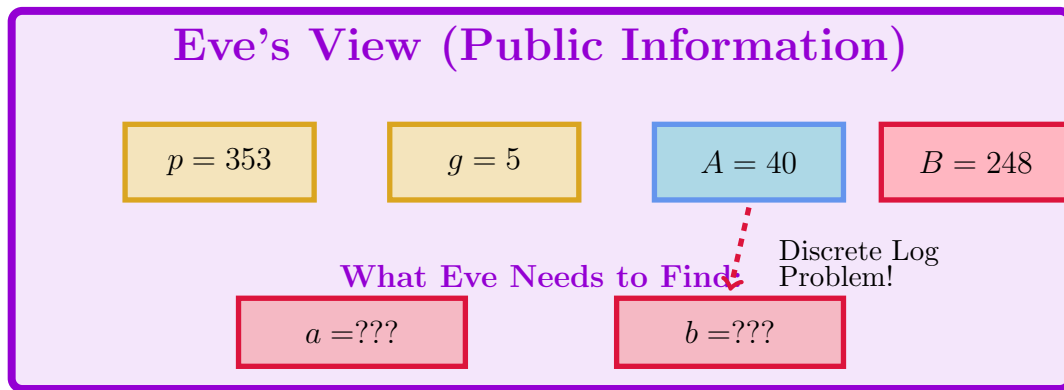


Figure 7: Eavesdropper's Perspective

4 Step-by-Step Algorithm

Algorithm 1 Diffie-Hellman Key Exchange Protocol

- 1: **SETUP (Public):**
 - 2: Choose a large prime number p
 - 3: Choose a primitive root g modulo p
 - 4: Both p and g are public knowledge
 - 5:
 - 6: **ALICE:**
 - 7: Generate random private key: $a \in \{2, 3, \dots, p-2\}$
 - 8: Compute public key: $A = g^a \bmod p$
 - 9: Send A to Bob over public channel
 - 10:
 - 11: **BOB:**
 - 12: Generate random private key: $b \in \{2, 3, \dots, p-2\}$
 - 13: Compute public key: $B = g^b \bmod p$
 - 14: Send B to Alice over public channel
 - 15:
 - 16: **ALICE COMPUTES SHARED SECRET:**
 - 17: $K_A = B^a \bmod p = (g^b)^a \bmod p = g^{ba} \bmod p$
 - 18:
 - 19: **BOB COMPUTES SHARED SECRET:**
 - 20: $K_B = A^b \bmod p = (g^a)^b \bmod p = g^{ab} \bmod p$
 - 21:
 - 22: **RESULT:**
 - 23: $K_A = K_B = K = g^{ab} \bmod p$ ✓
-

5 Mathematical Proof of Correctness

Why Both Get the Same Key

Alice's Computation:

$$\begin{aligned} K_A &= B^a \bmod p \\ &= (g^b \bmod p)^a \bmod p \\ &= g^{ba} \bmod p \quad (\text{by properties of modular exponentiation}) \end{aligned}$$

Bob's Computation:

$$\begin{aligned} K_B &= A^b \bmod p \\ &= (g^a \bmod p)^b \bmod p \\ &= g^{ab} \bmod p \quad (\text{by properties of modular exponentiation}) \end{aligned}$$

Conclusion:

$$K_A = g^{ba} \bmod p = g^{ab} \bmod p = K_B$$

Since multiplication is **commutative** ($ab = ba$), both parties compute the same shared secret!

6 Worked Example 1: Small Numbers (Educational)

Public Parameters

$$p = 23 \quad (\text{prime modulus})$$
$$g = 5 \quad (\text{primitive root of } 23)$$

6.1 Visual Step-by-Step Execution

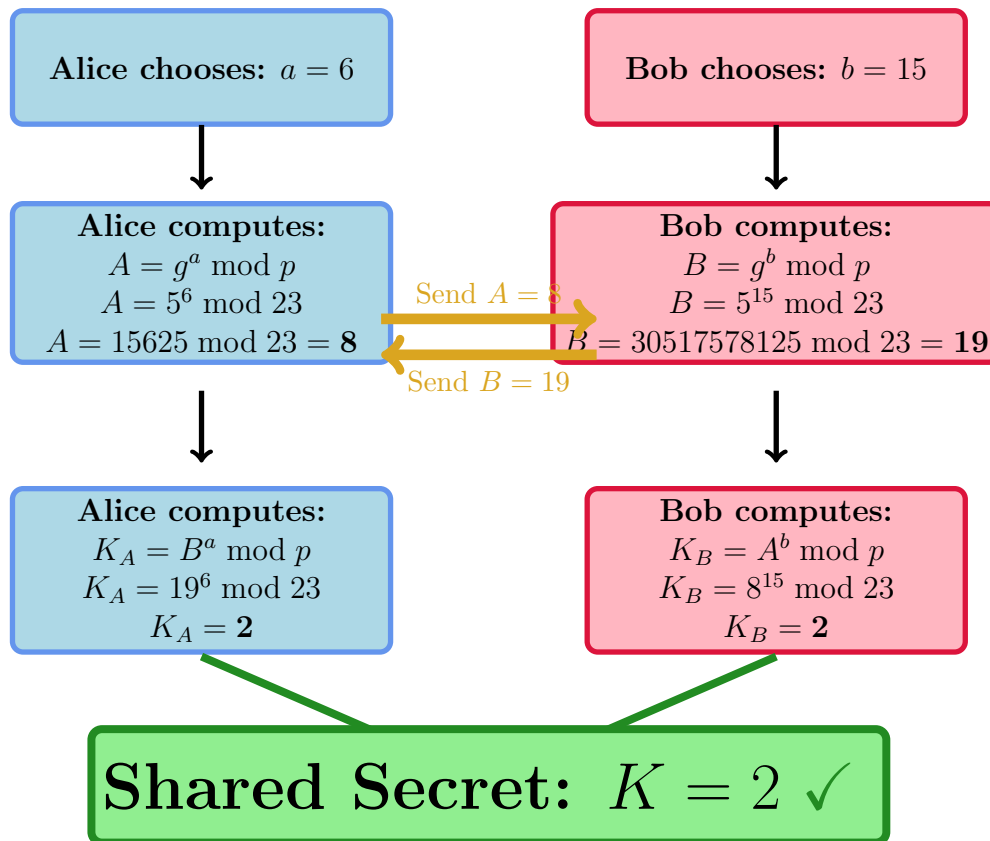


Figure 8: Example 1 Visual Execution

6.2 Detailed Calculation Breakdown

Computing $A = 5^6 \bmod 23$

Method: Repeated Squaring

$$5^1 = 5$$

$$5^2 = 25 \equiv 2 \pmod{23}$$

$$5^4 = (5^2)^2 = 2^2 = 4 \pmod{23}$$

$$5^6 = 5^4 \cdot 5^2 = 4 \cdot 2 = 8 \pmod{23}$$

Therefore: $A = 8$

Computing $B = 5^{15} \bmod 23$

Binary representation: $15 = 8 + 4 + 2 + 1$

$$5^1 = 5$$

$$5^2 = 2 \pmod{23}$$

$$5^4 = 4 \pmod{23}$$

$$5^8 = (5^4)^2 = 16 \pmod{23}$$

$$5^{15} = 5^8 \cdot 5^4 \cdot 5^2 \cdot 5^1$$

$$= 16 \cdot 4 \cdot 2 \cdot 5$$

$$= 640 \equiv 19 \pmod{23}$$

Therefore: $B = 19$

Alice computes:

$$\begin{aligned}K_A &= B^a \bmod p = 19^6 \bmod 23 \\19^2 &= 361 \equiv 16 \pmod{23} \\19^4 &= 16^2 = 256 \equiv 3 \pmod{23} \\19^6 &= 19^4 \cdot 19^2 = 3 \cdot 16 = 48 \equiv \mathbf{2} \pmod{23}\end{aligned}$$

Bob computes:

$$\begin{aligned}K_B &= A^b \bmod p = 8^{15} \bmod 23 \\8^2 &= 64 \equiv 18 \pmod{23} \\8^4 &= 18^2 = 324 \equiv 2 \pmod{23} \\8^8 &= 2^2 = 4 \pmod{23} \\8^{15} &= 8^8 \cdot 8^4 \cdot 8^2 \cdot 8^1 = 4 \cdot 2 \cdot 18 \cdot 8 \\&\equiv \mathbf{2} \pmod{23}\end{aligned}$$

Verification:

$$\boxed{K_A = K_B = 2}$$

7 Worked Example 2: Moderate Size

Public Parameters

$$\begin{aligned}p &= 97 \quad (\text{prime modulus}) \\g &= 5 \quad (\text{primitive root of } 97)\end{aligned}$$

Private key: $a = 36$

Public key computation:

$$A = g^a \bmod p = 5^{36} \bmod 97$$

Using binary method ($36 = 32 + 4$):

$$5^1 = 5$$

$$5^2 = 25$$

$$5^4 = 625 \equiv 43 \pmod{97}$$

$$5^8 = 43^2 = 1849 \equiv 87 \pmod{97}$$

$$5^{16} = 87^2 = 7569 \equiv 18 \pmod{97}$$

$$5^{32} = 18^2 = 324 \equiv 33 \pmod{97}$$

$$5^{36} = 5^{32} \cdot 5^4 = 33 \cdot 43 = 1419 \equiv \mathbf{50} \pmod{97}$$

$$\boxed{A = 50}$$

Private key: $b = 58$

Public key computation:

$$B = g^b \bmod p = 5^{58} \bmod 97$$

Binary: $58 = 32 + 16 + 8 + 2$

$$\begin{aligned}5^{58} &= 5^{32} \cdot 5^{16} \cdot 5^8 \cdot 5^2 \\&= 33 \cdot 18 \cdot 87 \cdot 25 \\&\equiv \mathbf{44} \pmod{97}\end{aligned}$$

$$\boxed{B = 44}$$

Alice computes:

$$K_A = B^a \bmod p = 44^{36} \bmod 97 = \mathbf{75}$$

Bob computes:

$$K_B = A^b \bmod p = 50^{58} \bmod 97 = \mathbf{75}$$

Verification:

$$K = 75 = g^{ab} \bmod p = 5^{2088} \bmod 97$$

8 Worked Example 3: Larger Prime

Public Parameters

$$p = 353 \quad (\text{prime modulus})$$

$$g = 3 \quad (\text{primitive root of } 353)$$

Private key: $a = 97$

Public key computation: $A = 3^{97} \bmod 353$

Binary representation: $97 = 64 + 32 + 1$

$$\begin{aligned} 3^1 &= 3 \\ 3^2 &= 9 \\ 3^4 &= 81 \\ 3^8 &= 6561 \equiv 212 \pmod{353} \\ 3^{16} &= 212^2 \equiv 349 \pmod{353} \\ 3^{32} &= 349^2 \equiv 228 \pmod{353} \\ 3^{64} &= 228^2 \equiv 206 \pmod{353} \\ 3^{97} &= 3^{64} \cdot 3^{32} \cdot 3^1 \\ &= 206 \cdot 228 \cdot 3 \\ &\equiv \mathbf{40} \pmod{353} \end{aligned}$$

$$A = 40$$

Private key: $b = 233$

Public key computation: $B = 3^{233} \bmod 353$

Binary: $233 = 128 + 64 + 32 + 8 + 1$

$$\begin{aligned} 3^{128} &= (3^{64})^2 \equiv 206^2 \equiv 288 \pmod{353} \\ 3^{233} &= 3^{128} \cdot 3^{64} \cdot 3^{32} \cdot 3^8 \cdot 3^1 \\ &= 288 \cdot 206 \cdot 228 \cdot 212 \cdot 3 \\ &\equiv \mathbf{248} \pmod{353} \end{aligned}$$

$$B = 248$$

Alice computes:

$$K_A = B^a \bmod p = 248^{97} \bmod 353 = \mathbf{160}$$

Bob computes:

$$K_B = A^b \bmod p = 40^{233} \bmod 353 = \mathbf{160}$$

Complete Verification:

$$(g^a)^b \equiv 40^{233} \equiv 160 \pmod{353}$$

$$(g^b)^a \equiv 248^{97} \equiv 160 \pmod{353}$$

$$g^{ab} = g^{97 \times 233} = g^{22601} \equiv 160 \pmod{353}$$

$$\boxed{K = 160}$$

9 Computational Complexity Analysis

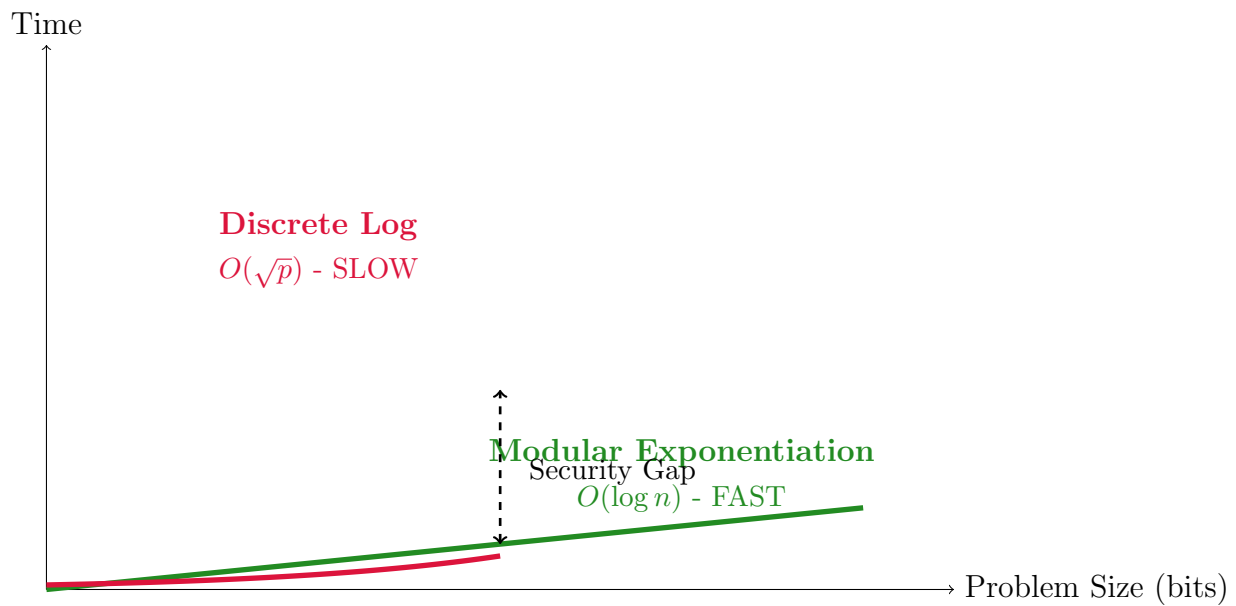


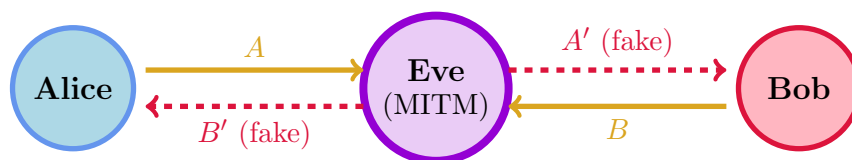
Figure 9: Computational Complexity Comparison

9.1 Performance Comparison Table

Bit Size	Modular Exp	Discrete Log	Security
512 bits	Milliseconds	Years	Weak
1024 bits	Milliseconds	Centuries	Moderate
2048 bits	< 1 second	> Age of Universe	Strong
3072 bits	< 2 seconds	Impossible	Very Strong

10 Security Analysis

10.1 Attack Scenarios



Solution: Authentication!

Figure 10: Man-in-the-Middle Attack

10.2 Security Requirements

Essential Security Measures

1. **Large Prime:** At least 2048 bits (3072+ recommended)
2. **Strong Generator:** Must be a primitive root modulo p
3. **Random Private Keys:** Use cryptographically secure RNG
4. **Authentication:** Combine with digital signatures (e.g., RSA, ECDSA)
5. **Perfect Forward Secrecy:** Use ephemeral keys (new for each session)

Vulnerabilities

- × No built-in authentication
- × Susceptible to Man-in-the-Middle attacks
- × Weak if small primes are used
- × Vulnerable to quantum computers (Shor's algorithm)

11 Real-World Applications

11.1 Modern Cryptographic Protocols

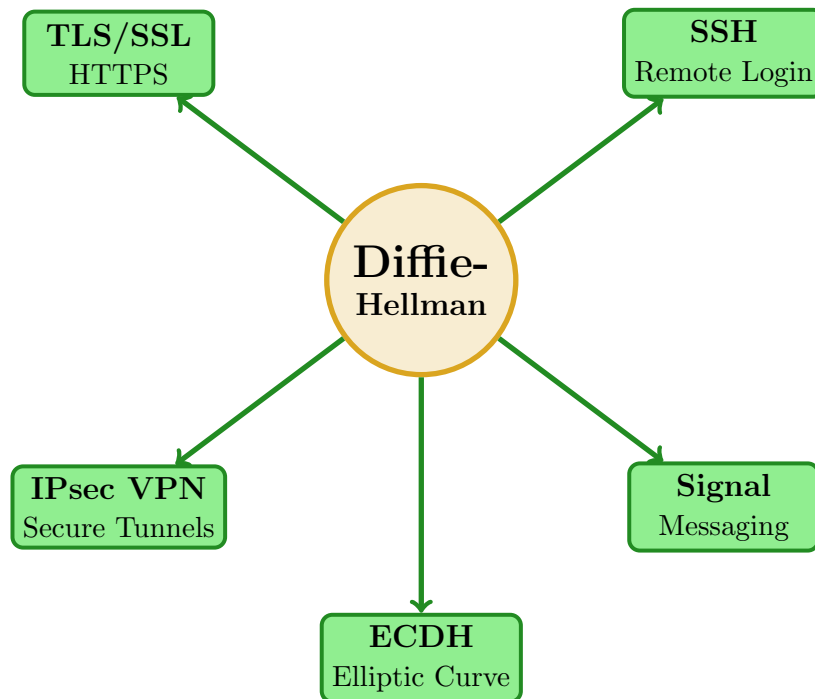


Figure 11: Diffie-Hellman in Modern Cryptography

11.2 TLS Handshake Visualization

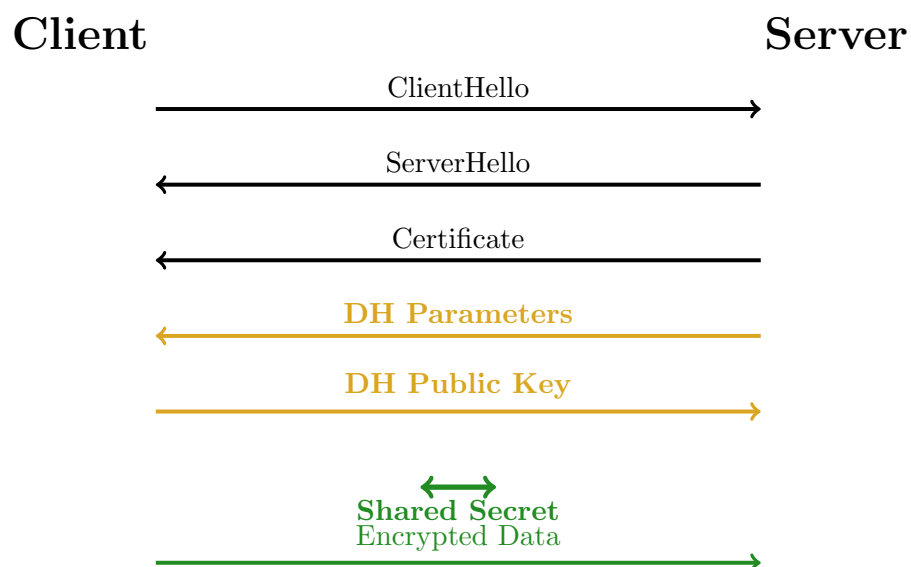


Figure 12: Diffie-Hellman in TLS Handshake

12 Elliptic Curve Variant (ECDH)

12.1 Why Elliptic Curves?

Advantages of ECDH

Smaller Keys, Same Security:

RSA/DH Key Size	ECC Key Size
1024 bits	160 bits
2048 bits	224 bits
3072 bits	256 bits

Benefits:

- Faster computation
- Less bandwidth
- Lower power consumption
- Perfect for mobile devices

12.2 Visual Comparison



Figure 13: DH vs ECDH Comparison

13 Summary and Key Takeaways

Essential Concepts

1. **Core Principle:** Exchange public values over insecure channel to establish shared secret
2. **Mathematical Foundation:** Based on difficulty of discrete logarithm problem
3. **Security:** Easy to compute forward, hard to reverse
4. **Limitation:** Vulnerable to MITM without authentication
5. **Solution:** Combine with digital signatures or certificates
6. **Modern Use:** Foundation for TLS, SSH, VPN, and secure messaging

The Magic Formula

$$K_{\text{Alice}} = (g^b)^a = g^{ab} = (g^a)^b = K_{\text{Bob}}$$

Both parties compute the same secret without ever transmitting it!

14 Practice Problems for Students

Exercise 1

Given $p = 37$, $g = 2$, Alice chooses $a = 5$, Bob chooses $b = 7$.
Find:

1. Alice's public key A
2. Bob's public key B
3. The shared secret K

Exercise 2

Explain why an eavesdropper who knows p , g , A , and B cannot easily compute the shared secret K .

Exercise 3

Given $p = 71$, $g = 7$, and $A = 51$:

1. Why is it difficult to find Alice's private key a ?
2. What attack could compromise this exchange?
3. How would you prevent this attack?

15 Further Reading and Resources

- **Original Paper:** Diffie, W., & Hellman, M. (1976). "New Directions in Cryptography"
- **RFC 2631:** Diffie-Hellman Key Agreement Method
- **RFC 3526:** More Modular Exponential (MODP) Diffie-Hellman groups
- **NIST Standards:** SP 800-56A - Recommendation for Pair-Wise Key Establishment

Questions or Discussion?

Contact: kmkhol101@gmail.com

Dr. Mohammed Tawfik
Assistant Professor of Cybersecurity and Cloud Computing
Ajloun National University, Jordan